

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)
Question Count: 5

Total Marks: 40

Code of Conduct

I **G. Ramya/192572073** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G. Ramya**

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of	20%	Concepts are	Mostly accurate	Partial application	Weak application

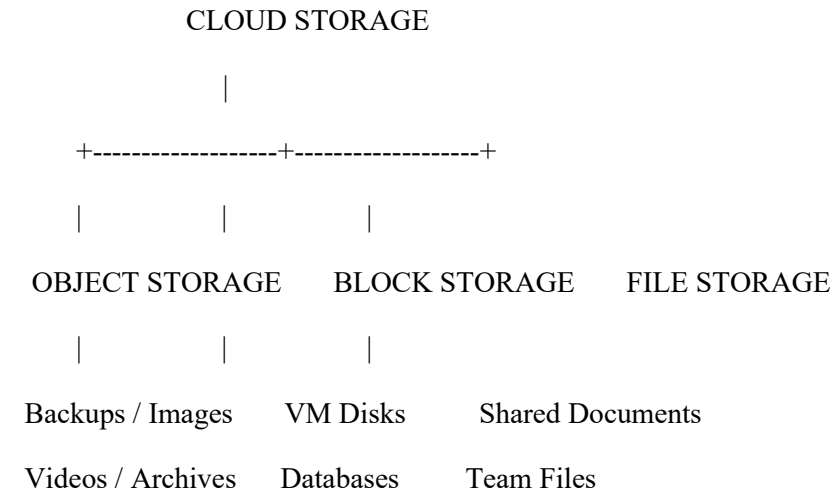
Concepts		applied accurately to practical tasks	application		
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

Experiment 1: Cloud Storage Comparison

Objective

To compare **object, block, and file storage** and select the most suitable storage model for backups, VM disks, and shared enterprise documents.

Architecture Sketch



Procedure

- 1. Identify the three major cloud storage models.
- 2. Study how each model stores and accesses data.
- 3. Compare their scalability, performance, accessibility, and use cases.
- 4. Match each storage model with the required application.
- 5. Select the most appropriate model for each requirement.

Comparison

Feature	Object Storage	Block Storage	File Storage
Storage unit	Object	Block	File
Access method	API/HTTP	Block-level	File system

Feature	Object Storage	Block Storage	File Storage
Scalability	Very high	High	High
Best for	Backups, media	VM disks, databases	Shared documents
Sharing	Through APIs	Limited	Excellent
Example	Amazon S3	Amazon EBS	Amazon EFS

Expected Output

- **Backups → Object Storage**
- **VM Disks → Block Storage**
- **Shared Enterprise Documents → File Storage**

Conclusion

Object storage is best for large-scale backups, block storage is best for high-performance VM disks, and file storage is best for shared enterprise documents. Selecting the correct storage model improves **performance, scalability, availability, and cost efficiency**.

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

Objective

To develop and demonstrate a simple **REST API workflow** for a cloud-based student portal, including request, response, authentication, and deployment.

Architecture Sketch

Student / Browser

|

| HTTP Request

↓

API Gateway

|

↓

REST API Server

|

↓

Authentication

|

↓

Database

|

↓

HTTP Response

|

↓

Student / Browser

Procedure

1. Create a student portal API.
2. Define API endpoints such as:
 - POST /login
 - GET /students
 - GET /students/{id}
 - POST /students
3. The student sends an HTTP request from a browser or application.
4. The API server receives and validates the request.
5. Authenticate the user using credentials or a token.
6. The API accesses the cloud database.
7. Generate a JSON response.
8. Deploy the API to a cloud server or serverless platform.
9. Test the API using a browser or API testing tool.

Request Example

GET /students/101

Authorization: Bearer <token>

Response Example

{

```
"student_id": 101,  
"name": "Rahul",  
"department": "CSE-AI",  
"status": "Active"  
}
```

Expected Output

Client



HTTP Request



Authentication



REST API



Cloud Database



JSON Response



Client

The student should receive the requested information in a structured JSON format.

Conclusion

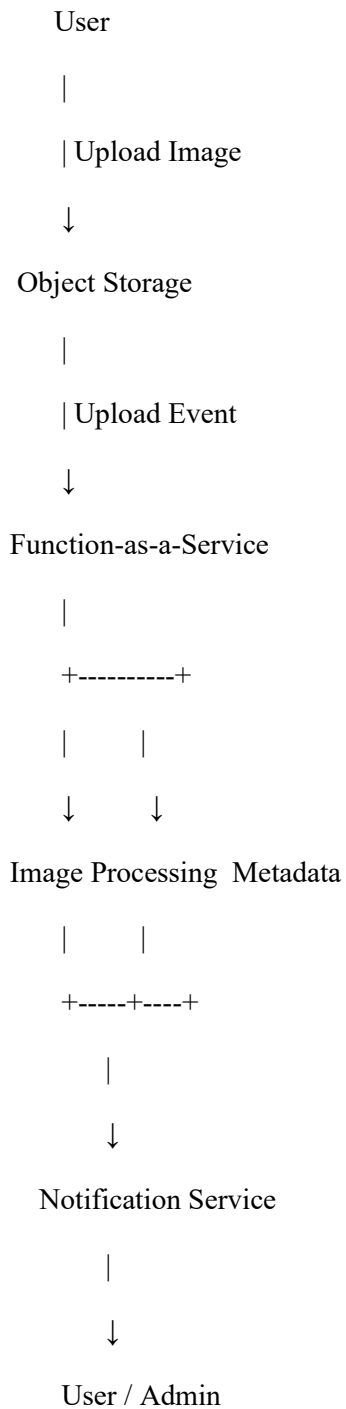
A REST API provides a simple and standardized method for communication between the student portal and cloud services. Authentication protects student information, while cloud deployment provides **scalability, availability, and accessibility**.

Experiment 3: Serverless Image Upload Processing

Objective

To design a **serverless event-driven workflow** for automatically processing images uploaded to cloud object storage.

Architecture Sketch



Procedure

1. Create a cloud object-storage bucket.
2. Configure the bucket to accept image uploads.
3. Configure an upload event trigger.
4. Create a serverless function.
5. Trigger the function whenever a new image is uploaded.
6. Perform processing such as:

- Image resizing
 - Thumbnail generation
 - Format conversion
 - Metadata extraction
7. Store the processed image back in object storage.
 8. Send a notification after successful processing.
 9. Monitor the function execution and errors.

Expected Output

When a user uploads:

photo.jpg

The system automatically produces:

Original Image

↓

Serverless Function

↓

thumbnail.jpg

↓

Notification: "Image processed successfully"

Conclusion

The serverless architecture automatically responds to image-upload events without requiring continuously running servers. It provides **automatic scaling, reduced infrastructure management, event-driven processing, and cost efficiency**.

Experiment 4: Edge vs Fog Computing for Smart Traffic Management

Objective

To compare **edge computing and fog computing** for a smart traffic management system based on architecture, latency, and deployment location.

Architecture Sketch

Edge Computing

Traffic Sensors / Cameras



Edge Device



Traffic Signal

Processing occurs very close to the sensors.

Fog Computing

Traffic Sensors



Edge Devices



Fog Gateway / Local Server



Multiple Traffic Signals



Cloud

Procedure

- 1. Identify traffic sensors, cameras, and traffic signals.
- 2. Determine which decisions require immediate responses.
- 3. Implement edge processing close to the traffic sensors.
- 4. Use fog nodes to aggregate information from multiple intersections.
- 5. Send summarized data to the cloud.
- 6. Compare latency, bandwidth consumption, and deployment location.
- 7. Evaluate which approach is suitable for different traffic-management operations.

Comparison

Factor	Edge Computing	Fog Computing
Processing location	Near sensor/device	Local network/gateway
Latency	Very low	Low
Coverage	Individual device/location	Multiple nearby devices

Factor	Edge Computing	Fog Computing
Bandwidth usage	Very low	Low
Best use	Immediate decisions	Local coordination
Example	Accident detection	Coordinating several intersections

Latency Consideration

For emergency traffic control, very low latency is required.

Sensor → Edge → Traffic Signal

is faster than:

Sensor → Internet → Cloud → Internet → Traffic Signal

Fog computing is useful when information from multiple traffic signals needs to be combined before making a decision.

Deployment Location

- **Edge:** Cameras, sensors, traffic lights, roadside devices.
- **Fog:** Local traffic-control centers, gateways, or nearby edge servers.
- **Cloud:** Centralized data center for long-term analytics and planning.

Expected Output

The system should:

- Detect traffic conditions quickly.
- Reduce response time.
- Reduce unnecessary cloud communication.
- Coordinate traffic signals efficiently.
- Send historical traffic information to the cloud.

Conclusion

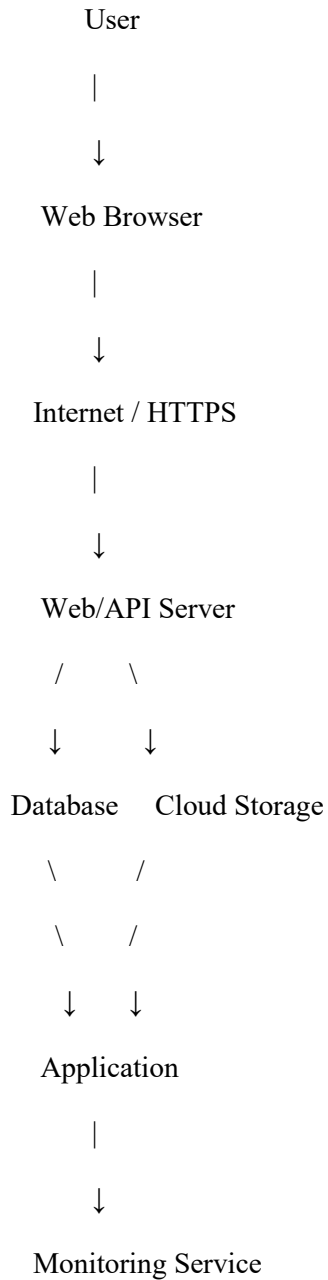
Edge computing is better for immediate, device-level decisions because it provides extremely low latency. **Fog computing** is better for coordinating multiple nearby devices. A combined **Edge–Fog–Cloud architecture** provides an effective solution for smart traffic management.

Experiment 5: Cloud Application Deployment Workflow

Objective

To demonstrate a complete cloud application workflow involving **client access, browser interaction, web API calls, storage access, and monitoring**.

Architecture Sketch



Procedure

1. Develop a simple cloud-based web application.
2. Deploy the application on a cloud platform.
3. Configure a public URL or domain.
4. Open the application using a web browser.
5. The browser sends an HTTPS request to the web server.

6. The web server calls the required REST API.
7. The API communicates with the database and cloud storage.
8. The application receives the required information.
9. The response is returned to the browser.
10. Configure monitoring to observe:
 - CPU and memory usage
 - API response time
 - Errors
 - Application availability
 - Storage usage

Workflow

User



Browser



HTTPS Request



Web/API Server



REST API



Database / Cloud Storage



Response



Browser



User

Monitoring Flow

Cloud Application



Monitoring Service



Metrics + Logs + Alerts



Administrator

Expected Output

The deployed application should successfully:

- Open through a browser.
- Accept user requests.
- Send web API calls.
- Retrieve data from cloud storage/database.
- Return results to the user.
- Generate monitoring metrics and logs.
- Alert administrators when problems occur.

Conclusion

A cloud application deployment connects the **client, browser, web/API layer, database, cloud storage, and monitoring services** into one integrated system. Monitoring ensures that performance and availability can be continuously observed, making the application more **reliable, scalable, and manageable**.